

Documento de Diseño Proyecto 2

- Juan Esteban Gomez
- Andres Gratz - 202514109
- Santiago Ismael Escobar Maidana- 202516956
- Juan Daniel Gonzalo López Pedraza - 202514683

Descripción de clases

Usuario

Representa a cualquier persona con acceso al sistema mediante credenciales.

Atributos:

- login: String
- password: String
- nombre: String
- juegosFavoritos: ArrayList<Juego>
- puntosFidelidad: int
- ventas: List<Venta>

Es una clase abstracta. No se instancian usuarios genéricos, solo tipos específicos como cliente, empleado o administrador.

Cliente

Representa al comensal que utiliza los servicios del café, realiza préstamos de juegos y consume productos.

Atributos:

- esNiño: boolean
- esJoven: boolean
- juegosReservados: int

Hereda de: Usuario

Responsabilidad: Reservar mesas, solicitar préstamos de juegos (máximo 2), comprar juegos o comida y acumular puntos de fidelidad.

Empleado

Representa al personal del café encargado de la operación diaria.

Atributos:

- codigoDescuento: String
- estáDeTurno: boolean
- turnos: List<Turno>

Hereda de: Usuario

Es una clase abstracta que se divide en roles operativos específicos.

Mesero

Tipo de empleado encargado de atender mesas, gestionar la caja y enseñar juegos difíciles.

Atributos:

- juegosConocidos: ArrayList<Juego>

Hereda de: Empleado

Responsabilidad: Atender pedidos, registrar ventas y brindar asesoría técnica en juegos etiquetados como "Difíciles".

Cocinero

Tipo de empleado encargado de la preparación de alimentos.

Hereda de: Empleado

Responsabilidad: Preparar los pedidos de pastelería y bebidas del menú.

Administrador

Usuario con facultades totales sobre la gestión financiera y de personal.

Hereda de: Usuario

Responsabilidad: Aprobar cambios de turno, gestionar inventarios (compras, reparaciones, bajas por robo) y generar informes de ventas segregados.

Turno

Representa la asignación horaria por día de los empleados, cada empleado puede tener muchos turnos a la semana.

Atributos:

- diaSemana: String

- estaAprobado: boolean

Responsabilidad: Organizar la jornada laboral y asegurar que se cumpla el personal mínimo requerido para operar.

Juego

Representa un título de mesa disponible en el catálogo del café.

Atributos:

- nombre: String
- anioPublicacion: int
- empresaMatriz: String
- minJugadores: int
- maxJugadores: int
- edadMinima: int
- categoria: String
- esDificil: boolean

copiaPrestamo

Representa una unidad física individual de un juego destinada exclusivamente al préstamo en sala.

Atributos:

- idUnico: String
- estado: String
- estaDisponible: Boolean

Responsabilidad: Mantener el registro individual del uso y el estado físico de cada copia del inventario de préstamo.

Mesa

Representa el espacio físico donde se ubican los clientes para consumir y jugar.

Atributos:

- idMesa: int
- capacidadMax: int
- ocupada: Boolean
- cantPersActuales: int
- hayNinos: boolean
- hayJovenes: boolean

Responsabilidad: Controlar la ocupación del local y validar que los juegos prestados sean aptos para los comensales presentes.

Prestamo

Representa la transacción temporal de uso de juegos de mesa por parte de un cliente o empleado.

Atributos:

- estado: String
- fechaHoraInicio: DateTime
- fechaHoraFin: DateTime

Responsabilidad: Registrar el historial de uso de los juegos, quién los tuvo y en qué tiempos.

Producto

Representa los elementos consumibles del menú de la cafetería.

Atributos:

- nombre: String
- precioBase: double

Es una clase abstracta para diferenciar entre tipos de productos.

Bebida

Tipo de producto líquido disponible en el menú.

Atributos:

- esAlcoholica: boolean
- esCaliente: boolean

Hereda de: Producto

Pasteleria

Tipo de producto sólido disponible en el menú.

Hereda de: Producto

Venta

Registro de una transacción comercial realizada en el establecimiento.

Atributos:

- idVenta: int
- fecha: Date
- subtotal: double
- impuestos: double
- propina: double
- total: double
- descuentoAplicado: int
- puntosGenerados: int

JuegoVenta

Representa el inventario de juegos destinados a la venta directa al público.

Atributos:

- precio: double
- stock: int

Responsabilidad: Gestionar la disponibilidad y el precio de los títulos comerciales, diferenciándolos del inventario de préstamo.

Torneo.java (Clase Abstracta)

Atributos: idTorneo (String), dia (DiaSemana), fechaInicio (Date), duracionMin (int), estado (EstadoTorneo), cupoTotal (int), cupoReservadoFanaticos (int), cupoReservadoOcupado (int), cupoOcupadoRegular (int), fechaCreacion (Date), nombre (String), juegoTorneo (Juego), inscripciones (List). Constante: REGLA_MAX_CUPOS__POR_USUARIO (int).

Hereda de: N/A (Es la clase base abstracta de la jerarquía).

Responsabilidad: Mantener el estado base de cualquier torneo. Controla la lógica fundamental matemática de cálculo y validación de cupos (regulares vs. fanáticos), la disponibilidad de plazas y define la plantilla para el comportamiento de inscripción/desinscripción que será extendida por sus subclasses.

TorneoAmistoso.java

Atributos: valorBono (double), bonoOtorgado (boolean), bonos (List).

Hereda de: Torneo.

Responsabilidad: Representar eventos puramente recreativos. Su responsabilidad principal es gestionar torneos que no tienen tarifa de entrada monetaria y cuya lógica de premiación consiste en generar y otorgar bonos de descuento a los ganadores al concluir el evento.

TorneoCompetitivo.java

Atributos: tarifaEntrada (double), pozoPremio (double), porcentajePozoPremio (double), pagosPorUsuario (Map<String, Double>).

Hereda de: Torneo.

Responsabilidad: Manejar eventos con premiación en dinero real. Se encarga de capturar y registrar el pago de la tarifa de entrada de cada participante, calculando progresivamente el monto final del "pozo acumulado" que se distribuirá al finalizar.

GestorTorneos.java

Atributos: catalogoTorneos (List), bonos (List).

Hereda de: N/A.

Responsabilidad: Actuar como el controlador principal (Manager) del módulo. Sus responsabilidades abarcan crear nuevos torneos verificando permisos de Administrador e inventario disponible, ejecutar la lógica de validación de inscripciones (cruce de horarios de empleados, límite de cupos), y procesar la finalización y cálculo de resultados/premios de los torneos.

InscripcionTorneo.java

Atributos: idInscripcion (String), fecha (Date), usuarios (List), cantidadCupos (int), cuposReservados (int), cuposRegulares (int), esEmpleado (boolean), montoPagado (double), pagoConfirmado (boolean), elegiblePrecioMetalico (boolean).

Hereda de: N/A.

Responsabilidad: Registrar el vínculo y la transacción de un participante (o grupo de participantes) hacia un torneo. Conserva la trazabilidad sobre de qué tipo de bolsas se sacaron los cupos (reservados o regulares), si se concretó el pago, y almacena las banderas de reglas de negocio como si es un empleado compitiendo (para negarle el premio en metálico).

BonoTorneoAmistoso.java

Atributos: codigo (String), valor (double), estado (String), fechaOtorgado (Date), fechaUsado (Date), ganador (Usuario), torneo (Torneo). Constantes de estado: ESTADO_DISPONIBLE, ESTADO_USADO.

Hereda de: N/A.

Responsabilidad: Modelar y persistir los detalles de un beneficio o cupón entregado como premio. Controla estrictamente su ciclo de vida y previene usos indebidos al registrar cuándo y en qué estado se encuentra (disponible vs. redimido).

ResultadoTorneo.java

Atributos: torneo (Torneo), ganador (Usuario), premioMetalico (double), bono (BonoTorneoAmistoso).

Hereda de: N/A.

Responsabilidad: Servir como un objeto contenedor inmutable (Data Transfer Object) para empaquetar toda la información de la resolución de un evento. Se utiliza como un registro final histórico de quién ganó y qué tipo de premio se llevó.

EstadoTorneo.java

Atributos: Los valores enumerados (ej. PROGRAMADO, EN_CURSO, FINALIZADO).

Hereda de: Enum.

Responsabilidad: Enumerar y limitar los estados lógicos válidos por los que transita el ciclo de vida cronológico y operativo de un torneo en la plataforma

Consola:

Este paquete forma la Capa de Presentación (Interfaz de Línea de Comandos - CLI). Su objetivo es interactuar directamente con el usuario, capturar sus entradas (inputs) por teclado y mostrar los menús y resultados del sistema.

Consola.java Es el punto de entrada principal de la aplicación. Se encarga de mostrar el menú inicial (login/registro), instanciar los gestores principales y redirigir el flujo de la aplicación hacia la consola específica según el rol del usuario autenticado (Cliente, Empleado o Administrador).

ConsolaAdministrador.java Interfaz dedicada al rol de Administrador. Presenta los menús y captura los datos necesarios para realizar tareas de alto nivel, como: crear torneos, comprar/reparar juegos, gestionar inventario, aprobar sugerencias de menú y generar reportes financieros.

ConsolaCliente.java Interfaz interactiva para los clientes del café. Les permite navegar por el catálogo de juegos, reservar mesas, solicitar préstamos de juegos, realizar compras en la tienda y gestionar su inscripción a torneos (amistosos o competitivos).

ConsolaEmpleado.java Interfaz diseñada para el personal (Meseros, Cocineros). Provee opciones operativas como consultar turnos asignados, solicitar intercambios de horario, sugerir nuevos platillos y atender las mesas o ventas regulares del local.

Relaciones del modelo

- **Relación entre Cliente y Mesa:** Un cliente debe ocupar una mesa para realizar préstamos de juegos. La mesa registra la cantidad de personas y la presencia de menores para validar la aptitud de los juegos y bebidas.

- **Relación entre Mesa y Juego:** Una mesa puede tener asociados simultáneamente hasta dos juegos del inventario de préstamos. Esta relación se bloquea si el juego no es apto para la edad de los comensales o el número de personas.
- **Relación entre Mesero y Juego:** Un mesero tiene una lista de juegos que conoce a cabalidad. Si un juego es "Difícil", se requiere idealmente un mesero con este conocimiento para realizar el préstamo.
- **Relación entre Empleado y Turno:** Cada empleado tiene asignado un turno semanal fijo. Los cambios o intercambios entre empleados requieren aprobación del administrador.
- **Relación entre Administrador e Inventarios:** El administrador gestiona dos inventarios separados: el de préstamo y el de venta. Puede mover juegos entre inventarios o darlos de baja por robo.
- **Relación entre Cliente y Préstamo:** Un cliente inicia un préstamo. El sistema debe registrar qué cliente es responsable de las copias mientras el préstamo esté activo.
- **Relación entre Préstamo y copiaPréstamo:** Un préstamo puede incluir de **1 a 2 copias** de juegos simultáneamente.
- **Relación entre Mesa y Préstamo:** Todo préstamo debe estar asociado a una mesa física para ser válido.
- **Relación entre Mesero y Préstamo:** Un mesero capacitado puede estar vinculado a un préstamo si el juego solicitado es "Difícil", para enseñar las reglas.

Reglas del dominio

1. Todo usuario debe poseer un login y un password para acceder al sistema.
2. Un cliente solo puede tener en préstamo un máximo de dos juegos simultáneamente.
3. El ingreso al café está limitado por una capacidad máxima; si se supera, la reserva de mesa es rechazada.
4. Los juegos de categoría "Acción" no pueden ser prestados a mesas que tengan bebidas calientes por motivos de seguridad.
5. Las bebidas alcohólicas no pueden ser despachadas a mesas donde se encuentren menores de 18 años.
6. Los juegos "Difíciles" pueden prestarse sin mesero capacitado solo bajo advertencia al cliente.
7. Las ventas de la tienda de juegos deben pagar un IVA del 19%.
8. Las ventas de cafetería deben pagar un impuesto al consumo del 8%.
9. Los empleados reciben un descuento del 20% en sus compras, y pueden compartir un 10% con otros clientes (no acumulable).
10. Para que el café opere legalmente, el turno debe contar con al menos 1 cocinero y 2 meseros simultáneamente.
11. Cada venta genera el 1% de su valor en puntos de fidelidad para el comprador, donde 1 punto equivale a 1 peso.

12. Los juegos que no se devuelven tras varios días deben marcarse como "Desaparecido" por el administrador.

Requerimientos Funcionales (RF)

Para Cliente

- **RF01 - Reservar Mesa:** El sistema debe permitir al cliente tomar una mesa indicando cuántas personas lo acompañan y si hay menores de edad.
- **RF02 - Consultar Catálogo:** El cliente debe poder visualizar los juegos disponibles para préstamo y venta.
- **RF03 - Solicitar Préstamo:** El cliente puede pedir hasta dos juegos simultáneamente si cumple con los requisitos de edad y capacidad de la mesa.
- **RF04 - Realizar Compra:** El cliente puede comprar juegos de la tienda o productos de cafetería.
- **RF05 - Gestionar Favoritos:** El cliente puede guardar una lista de sus juegos preferidos.

Para Empleado

- **RF06 - Consultar Turnos:** Los empleados pueden ver su horario semanal asignado.
- **RF07 - Solicitar Cambio de Turno:** Un empleado puede pedir un intercambio de turno con otro compañero o un cambio general.
- **RF08 - Sugerir Menú:** Cualquier empleado puede proponer nuevos platillos para el menú del café.
- **RF09 - Alquiler para Empleados:** Pueden alquilar juegos sin necesidad de mesa, siempre que no estén en su jornada laboral.
- **RF14 - Capacitación (Meseros):** Registrar y consultar los "Juegos Dificiles" en los que está capacitado para enseñar reglas.

Para Administrador

- **RF10 - Gestión de Inventario:** El administrador puede comprar, reabastecer, reparar o dar de baja juegos.
- **RF11 - Control de Personal:** El administrador debe aprobar o rechazar las solicitudes de cambio de turno.
- **RF12 - Generar Informes:** El sistema debe producir reportes de ventas detallados por rubro, fecha y desglose de impuestos/propinas.
- **RF13 - Actualizar Menú:** Potestad para agregar platillos o aceptar sugerencias de los empleados.

Nuevos Requerimientos Funcionales

RF15 - Registro autónomo de clientes

Descripción:

El sistema debe permitir que un cliente nuevo se registre por sí mismo en la aplicación para poder acceder a las funcionalidades del Board Game Café.

Actor principal: Cliente.

Entradas:

Login, contraseña, nombre, información de edad o clasificación del cliente.

Proceso:

El sistema valida que el login no exista previamente. Si el login está disponible, crea un nuevo objeto `Cliente` y lo registra dentro del gestor de usuarios.

Salida esperada:

El cliente queda registrado en el sistema y puede iniciar sesión con sus credenciales.

Restricciones:

No pueden existir dos usuarios con el mismo login.

RF16 - Registro de empleados por administrador

Descripción:

El sistema debe permitir que el administrador registre empleados dentro de la aplicación.

Actor principal: Administrador.

Entradas:

Login, contraseña, nombre, tipo de empleado y código de descuento.

Proceso:

El administrador ingresa los datos del empleado. El sistema valida que el login no esté repetido y crea el empleado correspondiente, ya sea mesero, cocinero u otro tipo permitido por el modelo.

Salida esperada:

El empleado queda registrado y puede iniciar sesión en la consola correspondiente.

Restricciones:

Solo el administrador puede registrar empleados. Los empleados no pueden registrarse por cuenta propia. El enunciado indica que los clientes sí deben poder registrarse autónomamente, mientras que los empleados deben ser registrados por el administrador.

RF17 - Autenticación obligatoria por tipo de usuario

Descripción:

El sistema debe exigir autenticación antes de permitir el uso de las funcionalidades de la aplicación.

Actores principales: Administrador, empleado y cliente.

Entradas:

Login y contraseña.

Proceso:

El sistema valida las credenciales ingresadas y determina el tipo real del usuario autenticado. Según su rol, se habilita la consola correspondiente.

Salida esperada:

El usuario accede únicamente a las funcionalidades permitidas para su tipo.

Restricciones:

Todos los usuarios deben autenticarse para utilizar las funcionalidades del sistema.

RF18 - Crear torneo amistoso

Descripción:

El sistema debe permitir que el administrador cree torneos amistosos asociados a un juego específico.

Actor principal: Administrador.

Entradas:

Juego, día de la semana, hora de inicio, número total de cupos y valor del bono de descuento.

Proceso:

El administrador selecciona el juego y define las características del torneo. El sistema valida que existan suficientes copias disponibles en el inventario de préstamos para soportar el número de participantes. Luego crea un objeto TorneoAmistoso.

Salida esperada:

El torneo amistoso queda registrado en el catálogo de torneos.

Restricciones:

Los torneos amistosos no cobran tarifa de entrada. El premio debe ser un bono de descuento redimible en una compra posterior.

RF19 - Crear torneo competitivo

Descripción:

El sistema debe permitir que el administrador cree torneos competitivos asociados a un juego específico.

Actor principal: Administrador.

Entradas:

Juego, día de la semana, hora de inicio, número total de cupos y tarifa de entrada.

Proceso:

El administrador define las características del torneo competitivo. El sistema valida que el juego exista y que el inventario de copias para préstamo permita cubrir el cupo total. Luego crea un objeto `TorneoCompetitivo`.

Salida esperada:

El torneo competitivo queda registrado en el catálogo de torneos.

Restricciones:

Los torneos competitivos cobran tarifa de entrada a los clientes y generan un premio metálico coherente con el dinero recaudado.

RF20 - Listar torneos por día

Descripción:

El sistema debe permitir consultar los torneos programados para un día específico de la semana.

Actores principales: Administrador, cliente y empleado.

Entradas:

Día de la semana.

Proceso:

El sistema busca dentro del catálogo de torneos todos los torneos cuyo día coincida con el ingresado.

Salida esperada:

Se muestra una lista de torneos con su ID, nombre, juego, tipo de torneo, cupos disponibles, día y hora.

Restricciones:

Puede haber más de un torneo simultáneamente en el café, por lo que el sistema debe poder listar múltiples torneos para un mismo día.

RF21 - Inscribir cliente a torneo

Descripción:

El sistema debe permitir que un cliente se inscriba a un torneo disponible.

Actor principal: Cliente.

Entradas:

ID del torneo y cantidad de cupos solicitados.

Proceso:

El sistema valida que el torneo exista, que haya cupos disponibles y que la cantidad solicitada no supere el máximo permitido por usuario. Si el cliente es fanático del juego, se intenta usar primero la bolsa de cupos reservados.

Salida esperada:

Se crea una inscripción asociada al cliente y al torneo. Los cupos ocupados del torneo se actualizan.

Restricciones:

Un usuario puede tomar máximo 3 cupos por torneo. Para cada par usuario-torneo solo debe existir una inscripción activa.

RF22 - Desinscribir cliente de torneo

Descripción:

El sistema debe permitir que un cliente cancele su inscripción a un torneo.

Actor principal: Cliente.

Entradas:

ID del torneo.

Proceso:

El sistema busca la inscripción del usuario en el torneo indicado. Si existe, elimina la inscripción y libera todos los cupos que habían sido tomados.

Salida esperada:

La inscripción desaparece del torneo y los cupos vuelven a estar disponibles.

Restricciones:

Al desinscribirse se deben eliminar todos los cupos tomados por ese usuario en ese torneo, no solo una parte.

RF23 - Manejar cupos reservados para fanáticos

Descripción:

El sistema debe reservar una parte de los cupos de cada torneo para los clientes fanáticos del juego.

Actores principales: Cliente y sistema.

Entradas:

Juego del torneo, favoritos del cliente y cantidad de cupos solicitados.

Proceso:

El sistema calcula el 20% de los cupos totales del torneo, redondeado hacia arriba. Si el cliente tiene el juego del torneo dentro de sus favoritos, se considera fanático y se le asignan cupos reservados mientras haya disponibilidad. Si los cupos reservados se agotan, puede tomar cupos regulares.

Salida esperada:

Los cupos se descuentan correctamente de la bolsa reservada o regular.

Restricciones:

El 20% de cupos reservados debe redondearse hacia arriba. Los fanáticos no quedan bloqueados si se acaban los cupos reservados; pueden tomar cupos regulares.

RF24 - Inscribir empleado a torneo

Descripción:

El sistema debe permitir que un empleado se inscriba a un torneo siempre que no esté de turno el día del evento.

Actor principal: Empleado.

Entradas:

ID del torneo y cantidad de cupos.

Proceso:

El sistema valida que el empleado no tenga turno asignado para el día del torneo. Si no hay conflicto, se permite la inscripción.

Salida esperada:

El empleado queda inscrito en el torneo.

Restricciones:

Un empleado no puede inscribirse si está cubriendo turno el día del torneo. En torneos competitivos, la inscripción de empleados es gratuita, pero no son elegibles para premio metálico.

RF25 - Registrar pago de torneo competitivo

Descripción:

El sistema debe registrar el pago de entrada de los clientes inscritos en torneos competitivos.

Actores principales: Cliente y sistema.

Entradas:

Usuario, torneo competitivo y monto de la tarifa.

Proceso:

Cuando un cliente se inscribe a un torneo competitivo, el sistema registra el monto pagado. El proceso de pago externo no se implementa, pero el sistema debe conservar la información del pago para calcular el premio.

Salida esperada:

La inscripción queda marcada con monto pagado y pago confirmado.

Restricciones:

No es necesario implementar pasarelas de pago, ya que el proceso está tercerizado. Los empleados no pagan tarifa de entrada en torneos competitivos.

RF26 - Finalizar torneo

Descripción:

El sistema debe permitir que el administrador finalice un torneo y registre un ganador.

Actor principal: Administrador.

Entradas:

ID del torneo y login del ganador.

Proceso:

El sistema valida que el torneo exista y que el ganador sea un usuario registrado. Luego cambia el estado del torneo a finalizado y calcula el resultado según el tipo de torneo.

Salida esperada:

Se genera un `ResultadoTorneo` con el ganador y el premio correspondiente.

Restricciones:

Solo el administrador puede finalizar torneos.

RF27 - Otorgar bono por torneo amistoso

Descripción:

El sistema debe otorgar un bono de descuento al ganador de un torneo amistoso.

Actores principales: Administrador, ganador y sistema.

Entradas:

Torneo amistoso finalizado y usuario ganador.

Proceso:

Al finalizar el torneo amistoso, el sistema crea un bono con código, valor, estado disponible, fecha de otorgamiento, ganador y torneo asociado.

Salida esperada:

El ganador recibe un bono disponible para usar en una compra futura.

Restricciones:

El bono no es acumulable con otros descuentos. El enunciado establece que el premio de descuento debe poder guardarse y usarse en una compra posterior.

RF28 - Calcular premio metálico de torneo competitivo

Descripción:

El sistema debe calcular el premio metálico de un torneo competitivo con base en el dinero recaudado por inscripciones.

Actores principales: Administrador, ganador y sistema.

Entradas:

Torneo competitivo finalizado y usuario ganador.

Proceso:

El sistema calcula el pozo de premio usando los pagos registrados en las inscripciones. Si el ganador es un cliente, se le asigna el premio metálico. Si el ganador es empleado, el premio metálico debe ser cero.

Salida esperada:

Se genera un resultado con el premio metálico correspondiente.

Restricciones:

Los empleados pueden participar en torneos competitivos gratis, pero no pueden recibir premio metálico.

RF29 - Usar bono de torneo en una compra

Descripción:

El sistema debe permitir que un usuario ganador de torneo amistoso use su bono de descuento en una compra posterior.

Actor principal: Cliente o empleado ganador.

Entradas:

Código del bono y venta a realizar.

Proceso:

El sistema valida que el bono exista, que esté disponible y que pertenezca al usuario

correspondiente. Luego aplica el descuento sobre la venta y cambia el estado del bono a usado.

Salida esperada:

La venta queda con el descuento aplicado y el bono queda marcado como usado.

Restricciones:

El bono no puede acumularse con otros descuentos ni con puntos de fidelidad si el sistema los maneja como descuento.

RF30 - Persistir torneos, inscripciones y bonos

Descripción:

El sistema debe guardar y cargar la información relacionada con torneos para mantener el estado entre ejecuciones.

Actores principales: Sistema.

Entradas:

Catálogo de torneos, inscripciones, pagos y bonos.

Proceso:

Al iniciar la aplicación, el sistema carga automáticamente los datos desde archivos predeterminados. Al finalizar, guarda los cambios realizados durante la ejecución.

Salida esperada:

Los torneos, inscripciones y bonos se conservan entre ejecuciones.

Restricciones:

No se debe pedir al usuario el nombre de los archivos para cargar o guardar. Los datos deben cargarse automáticamente y persistirse al terminar el programa.

RF31 - Validar entradas en consola

Descripción:

El sistema debe validar las entradas ingresadas por los usuarios en las consolas de administrador, cliente y empleado.

Actores principales: Administrador, cliente y empleado.

Entradas:

Opciones de menú, números enteros, valores monetarios, días de la semana, horas, login, contraseña e IDs de torneo.

Proceso:

El sistema verifica que los datos ingresados tengan el formato correcto y que correspondan a valores válidos antes de ejecutar cada acción.

Salida esperada:

Si la entrada es válida, se ejecuta la funcionalidad solicitada. Si no es válida, se muestra un mensaje de error apropiado.

Restricciones:

Las interfaces por consola deben permitir realizar todas las actividades necesarias según el tipo de usuario y deben validar todas las entradas recibidas.

RF32 - Consultar resultado de torneo

Descripción:

El sistema debe permitir consultar el resultado de un torneo finalizado.

Actores principales: Administrador, cliente y empleado.

Entradas:

ID del torneo.

Proceso:

El sistema busca el torneo finalizado y muestra el ganador y el premio otorgado, ya sea bono de descuento o premio metálico.

Salida esperada:

Se muestra el resultado del torneo de forma clara.

Restricciones:

Solo debe mostrarse resultado si el torneo ya fue finalizado.

Restricciones del Proyecto

Restricciones de Dominio (Reglas de Negocio)

- **Capacidad del Local:** No se permiten más personas de las que soporta la capacidad máxima definida.
- **Seguridad y Edad:** No se despacha alcohol a mesas con menores de 18 años.
- **Seguridad de Juegos:** Está prohibido servir bebidas calientes en mesas que tengan juegos de la categoría "Acción".
- **Mínimo de Personal:** Cada turno debe contar con al menos 1 cocinero y 2 meseros.
- **Impuestos:** Los juegos de mesa graban un 19% de IVA, mientras que la cafetería graba un 8% de impuesto al consumo.

Restricciones Técnicas

- **Persistencia:** Toda la información debe almacenarse en archivos fuera de la carpeta del código fuente.
- **Tecnología:** Desarrollo en lenguaje Java.
- **Acceso:** Todo usuario requiere login y password para interactuar con el sistema.
- **Presupuesto:** El desarrollo del proyecto se puede ver limitado por el presupuesto que tiene la empresa destinado para el desarrollo. Si hay poco presupuesto es probable que se tengan que sacrificar funcionalidades y/o mantenibilidad.

Descripción de los Programas de Prueba

Se implementarán los siguientes programas de prueba para probar el programa.

Prueba de Carga: Un programa que cargue datos desde los archivos y demuestre que la información se mantiene entre ejecuciones.

Prueba de Ventas y Puntos: Un programa que realice una compra y muestre cómo se acumulan los puntos de fidelidad en el perfil del usuario.

Prueba de Gestión de Inventario: Un programa donde el administrador marque un juego como "robado" y se verifique que ya no aparece disponible para préstamo.

Diseño e implementación

Introducción

En esta segunda fase, **implementamos** la lógica de negocio necesaria para transformar el análisis inicial en una aplicación funcional. El sistema no solo gestiona transacciones, sino

que garantiza el cumplimiento de reglas críticas como el control de aforo, la seguridad en el consumo de productos y la persistencia de datos. **Decidimos** priorizar la integridad de la información y la extensibilidad del código mediante una arquitectura orientada a objetos robusta.

2. Modelo de Diseño

2.1. Justificación de Decisiones Arquitectónicas

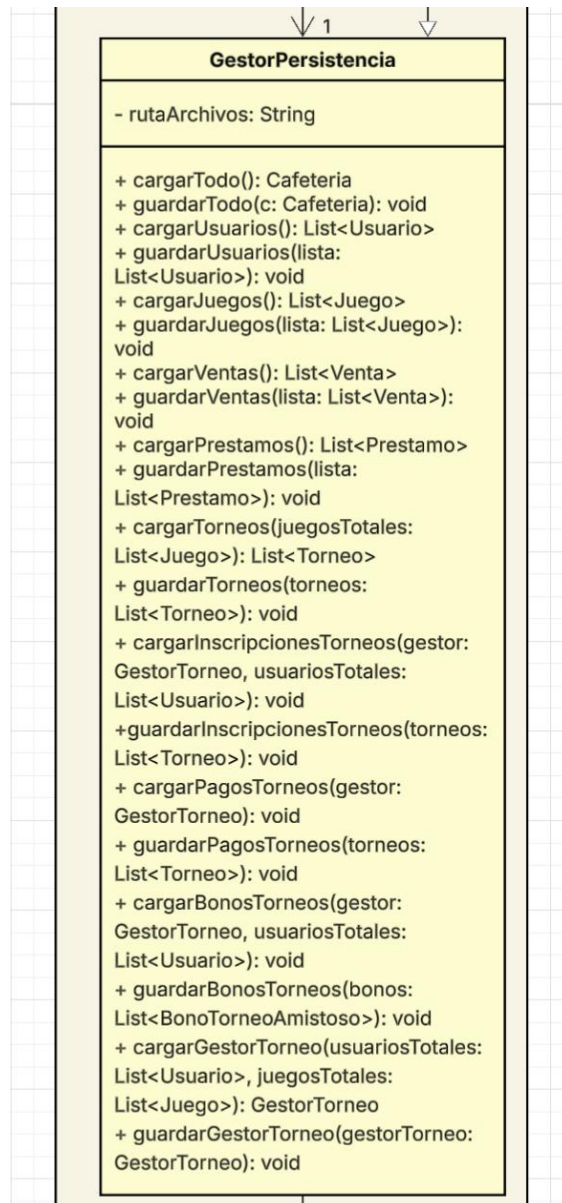
Tras analizar la retroalimentación de la Entrega 1, **realizamos cambios profundos** en la estructura:

- **Clase Central Cafeteria:** Implementamos esta clase como el corazón del sistema. Esta clase paso a ser el centro de todo el cual redirecciona y de el parten las conexiones a todo el flujo del sistema. Decidimos que esta clase sea la encargada de gestionar el aforo máximo (capacidadMax) y la ocupación actual. Al centralizar esto, garantizamos que ningún cliente ingrese si el establecimiento está lleno, cumpliendo con la restricción de seguridad.
- **Refactorización de Juegos y Copias:** Aunque inicialmente el profesor sugirió unificar, **decidimos** mantener una distinción clara en el diseño de bajo nivel entre CopiaPrestamo y CopiaVenta. **Analizamos** que la lógica de un préstamo (circular, con estados de "incompleto" o "desgastado") es intrínsecamente distinta a la de una venta a un cliente (salida definitiva del inventario). Sin embargo, ambas se asocian a la clase Juego para no duplicar la información técnica (autor, edad mínima, etc.). Sin embargo mantuvimos la posibilidad de que una copia de un juego que estaba en préstamo pase a ser una copia disponible para venta.
- **Jerarquía de Usuarios:** Implementamos una estructura de herencia donde Usuario es abstracta. Esto nos permitió aplicar polimorfismo en los procesos de login y asegurar que solo los Meseros puedan acceder a funciones de atención, mientras que el Administrador gestiona el inventario.
- **Control de Seguridad en Mesas:** Decidimos que la clase Mesa guarde el estado demográfico (hayNinos, hayJovenes). Esto permite que, antes de registrar un ItemVenta de tipo alcohol, el sistema verifique el estado de la mesa y bloquee la transacción si es necesario así como es solicitado en las reglas de negocio.
- **Consistencia de Atributos y Métodos:** Decidimos que cada mensaje en nuestros diagramas de secuencia (como actualizarPuntosFidelidad o estaDisponible) se convierta en un método public en Java, asegurando una trazabilidad del 100% entre el diseño y el código final.
- **Gestores:** Decidimos que existieran clases intermedias llamadas gestores, los cuales permiten que el flujo de la aplicación no sea descargado completamente sobre la clase cafetería, ya que esto violaría las reglas de diseño. Descargamos alguna lógica Importante sobre estos gestores, ya que permite tener mucho más desglosado el

código y evitar que la clase cafeteria quede supercargada, tener un flujo más limpio y tener los roles de diseño mucho más claros para cada clase.

3. Arquitectura de Persistencia

Implementamos un sistema de persistencia basado en archivos de texto plano (CSV), cumpliendo con la restricción de mantener los datos fuera de la carpeta src. Escogimos csv ya que existe bastante información en internet de como usar las librerías y implementar este tipo de persistencia, además permite editar fácilmente la información desde otro programa.

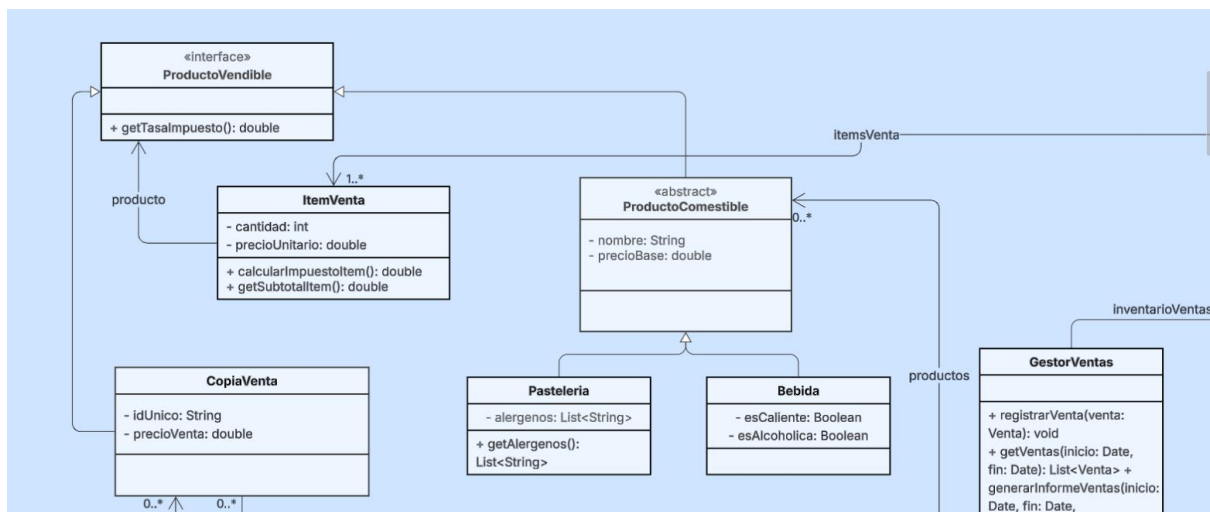


- **Estructura:** Creamos la clase **GestorPersistencia** encargada exclusivamente de la lectura y escritura.

- **Carga Inicial:** Al arrancar, cargarTodo() reconstruye el estado completo de la Cafetería, incluyendo la lista de usuarios, el catálogo de juegos y el historial de ventas desde la última sesión.
- **Relaciones:** Para persistir relaciones (ej. qué juegos tiene reservados un cliente), **decidimos** guardar los IDs de los juegos en el archivo de usuarios y, en tiempo de ejecución, "re-vincular" los objetos.

4. Lógica de Impuestos y Finanzas

Para cumplir con los requerimientos funcionales del negocio, **diseñamos** la interfaz ProductoVendible:

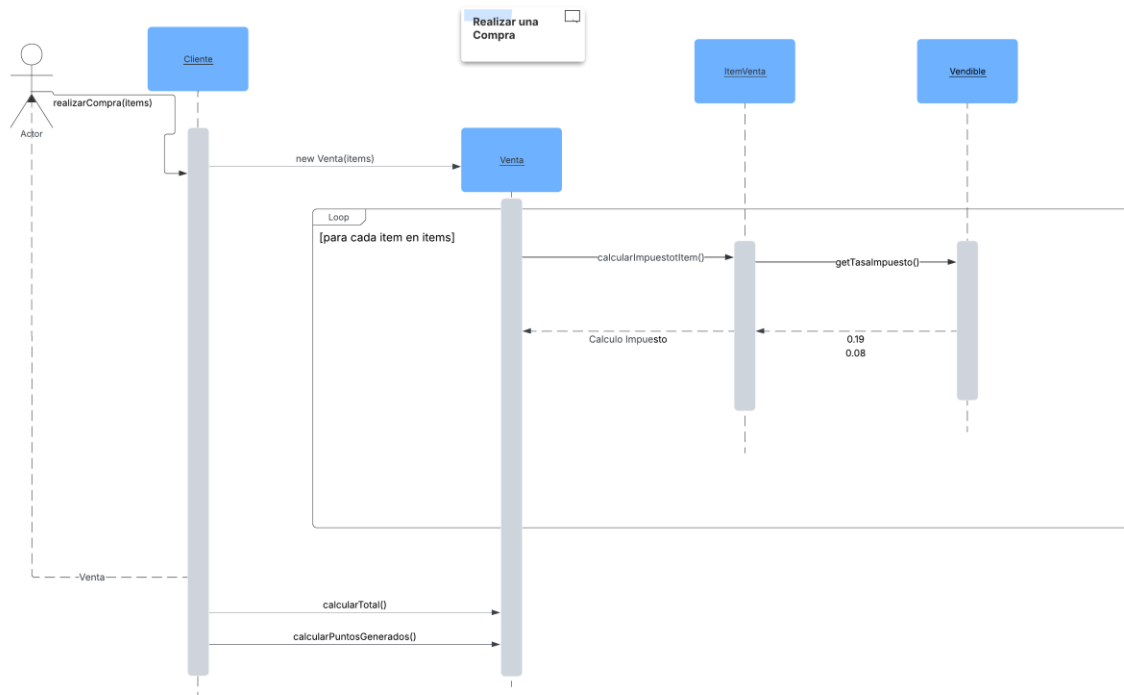


- **IVA (19%):** Aplicado automáticamente a través de la clase Juego y sus copias de venta.
- **Impuesto al Consumo (8%):** Aplicado a ProductoComestible (Bebidas y Pastelería).
- **Cálculo Automatizado:** El método `calcularImpuestosTotales()` en la clase Venta recorre los **ItemVenta**, identifica el tipo de producto y suma los gravámenes correspondientes, generando un desglose legal en la factura. Esto nos permitiera ser efectivos con los cálculos y el desglose de los impuestos/descuentos aplicados en una venta.

5. Diagramas de Secuencia (Validación de Lógica de Negocio)

Para asegurar que nuestro diseño de bajo nivel no solo fuera una estructura estática, sino un sistema capaz de soportar las reglas dinámicas del enunciado, **decidimos modelar** los siguientes tres flujos críticos:

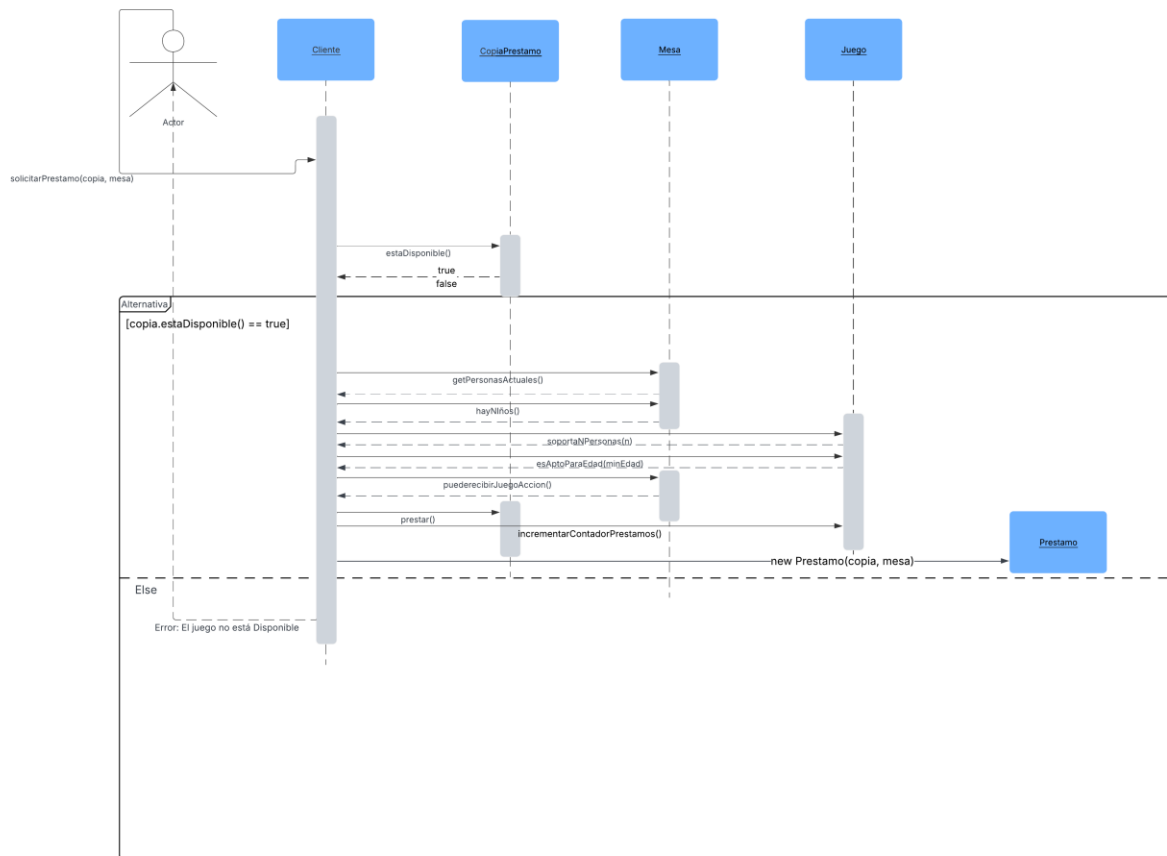
5.1. Realizar una Compra (Seguridad y Fidelización)



En este flujo, **implementamos** el rol del GestorVentas como el cerebro de la transacción.

- **Justificación de Diseño:** Analizamos que las restricciones de seguridad (como no vender alcohol a menores) no dependen solo de quién compra, sino del entorno (la mesa). Por ello, el diagrama muestra cómo el GestorVentas consulta a la Mesa los métodos `hayNinos()` y `hayJovenes()` antes de proceder.
- **Impacto en el Código:** Solo si estas validaciones pasan, se instancia el objeto `Venta`. Además, **decidimos** integrar aquí el sistema de puntos, permitiendo que el Cliente use sus beneficios (`usarPuntosFidelidad`) y acumule nuevos en la misma operación, garantizando la consistencia del estado financiero.

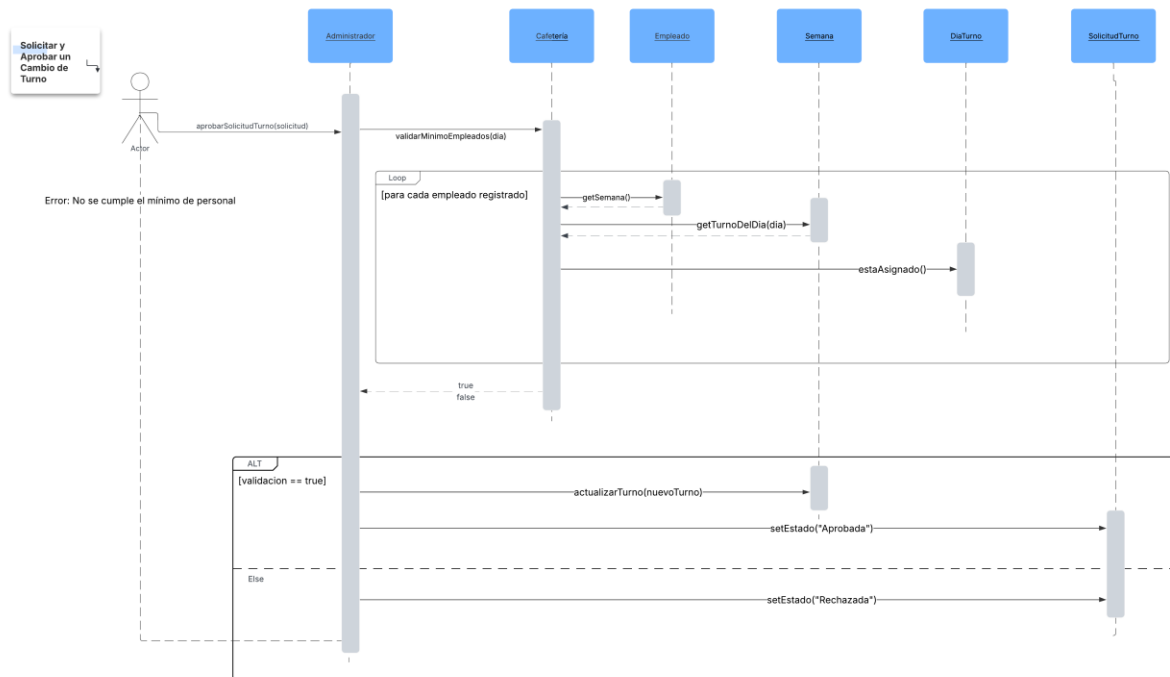
5.2. Préstamo de un Juego (Control de Inventario y Límites)



Este diagrama describe cómo gestionamos el recurso más valioso del café: los juegos de mesa.

- **Justificación de Diseño:** Para cumplir con la restricción de "máximo 2 juegos por cliente", decidimos que la Cafetería sea la encargada de validar el estado del usuario antes de permitir el préstamo.
- **Lógica Implementada:** El flujo detalla una doble validación: primero, verificamos la capacidad del cliente (`getJuegosReservados() < 2`) y luego la disponibilidad física del juego (`copia.estaDisponible()`). Solo al cumplirse ambas, se crea el objeto `Prestamo` y se actualiza el contador del cliente, evitando fugas de inventario o abusos en el servicio.

5.3. Solicitar y Aprobar un Cambio de Turno (Gestión Operativa)



Este es un flujo avanzado que **decidimos incluir** para demostrar que nuestro sistema soporta la gestión de personal exigida (mínimo 1 cocinero y 2 meseros).

- **Justificación de Diseño:** Analizamos que los turnos no pueden ser modificados de forma aislada por los empleados. Por eso, introducimos la clase Semana como el validador de horarios.
- **Flujo de Trabajo:** Un Empleado inicia la solicitud, pero el intercambio solo se concreta si el otro empleado acepta y si la Semana ejecuta el método `actualizarTurno()`. Esto nos permite, en la implementación, incluir una lógica que verifique que el cambio no deje al café sin el personal mínimo requerido para operar.

6. Documentación de Programas de Prueba

Analizamos los errores pasados y **decidimos** formalizar las pruebas con el siguiente rigor:

Descripción de los Programas de Prueba

Se implementarán los siguientes programas de prueba para probar el programa:

Pruebas de flujo:

1. **Prueba de Flujo 1: Restricciones cruzadas de edad, menú y categorías de juego.**

Cubre:

- Verificación de aforo al asignar una mesa.
- Rechazo de bebidas alcohólicas en mesas con menores.
- Rechazo de juegos de Acción cuando hay bebidas calientes en la mesa.
- Aprobación del préstamo de un juego de Tablero apto para todo público.

2. Prueba de Flujo 2: Juegos "Difíciles", compras en tienda y fidelización.

Cubre:

- Préstamo de juego difícil sin mesero que lo conozca → advertencia.
- Compra de copia desde el inventario de ventas (separado del de préstamos).
- Aplicación de descuento por código de empleado (10%).
- Cobro de IVA del 19% sobre la copia de juego vendida.
- Acumulación de 1% del total como puntos de fidelidad.

3. Prueba de Flujo 3: Administración extrema: (Robos, reparación y reportes financieros)

Cubre:

- Marcar copia como desaparecida (robada).
- Mover copia de inventario de ventas al de préstamos (reparación de inventario).
- Informe de ventas que segrega: IVA 19% (juegos) vs Consumo 8% + Propina 10% (comida).

4. Prueba de Flujo 4: Lógica de empleados (Turnos y alérgenos).

Cubre:

- Solicitud de cambio de turno por parte de un mesero.
- Rechazo automático del cambio si viola el mínimo de empleados en turno.
- Sugerencia de nuevo platillo por un cocinero.
- Aprobación de la sugerencia y registro del alérgeno por el administrador.

Pruebas para Proyecto 2:

1. PruebaRegistroCliente.java

Objetivo: Asegurar que un nuevo cliente pueda registrarse e iniciar sesión por sí mismo.

¿Qué hace? Invoca la creación de un nuevo cliente a través de GestorUsuarios, verificando que se almacena en la lista global del sistema y que, al utilizar las credenciales correctas ("juan123" y "1234"), la autenticación devuelve un objeto usuario válido en lugar de un error o valor nulo.

2. PruebaLimiteCuposTorneo.java

Objetivo: Verificar la regla de negocio que prohíbe inscribir a más de 3 personas a la vez.

¿Qué hace? Simula un intento de tomar 4 cupos para un torneo amistoso. Luego, verifica mediante el validador del sistema que la acción es bloqueada (retornando false) y comprueba rigurosamente que los cupos totales del torneo permanecen inalterados.

3. PruebaDescuentoCuposRegulares.java

Objetivo: Comprobar la correcta reducción de cupos cuando un cliente común se inscribe.

¿Qué hace? Toma un torneo con 10 cupos en total (8 regulares, 2 reservados). Un cliente no fanático se inscribe usando 2 cupos regulares. La prueba aserta que los cupos regulares disponibles bajen de 8 a 6 de forma precisa, validando que el proceso matemático de la inscripción funciona sin arrojar excepciones.

4. PruebaReglaFanaticosTorneo.java

Objetivo: Probar el manejo exclusivo del 20% de cupos para clientes fanáticos.

¿Qué hace? Se inscribe a un torneo a un cliente que tiene marcado el juego principal como "Favorito". El test verifica que el sistema lo identifique como fanático y evalúa que el cupo consumido se descuenta de la bolsa reservada (reduciéndola de 2 a 1) dejando intactos los 8 cupos regulares.

5. PruebaDesinscripcionTorneo.java

Objetivo: Validar que al cancelar una inscripción, los cupos regresan a estar disponibles.

¿Qué hace? Registra una inscripción inicial ocupando 3 cupos regulares. Posteriormente, manda a cancelar dicha inscripción. La prueba valida que el registro desaparece de la lista de inscritos y que los 3 cupos consumidos regresan al conteo de "disponibles" del torneo.

6. PruebaCreacionTorneoAmistoso.java

Objetivo: Comprobar la instanciación de un Torneo Amistoso.

¿Qué hace? Simula la creación de un torneo amistoso por parte de un Administrador. Asegura que el objeto resultante sea efectivamente del tipo TorneoAmistoso y consolida que atributos como la tarifa (implícita en 0) y su configuración para otorgar bonos de premio operen bajo su respectiva clase.

7. PruebaCreacionTorneoCompetitivo.java

Objetivo: Confirmar que los torneos competitivos manejan las tarifas de entrada.

¿Qué hace? Por medio de Reflexión en Java, valida la existencia de la clase y la capacidad del sistema para instanciar un Torneo Competitivo, el cual está orientado a cobrar una tarifa en dinero a cada participante (elemento clave para el posterior pozo de premios).

8. PruebaBloqueoTurnoTorneo.java

Objetivo: Impedir que los empleados participen en torneos cuando deben estar trabajando.

¿Qué hace? A un mesero se le asigna un turno el día Viernes. Luego, el mesero intenta inscribirse a un torneo programado también para el Viernes. El test detecta la colisión del día asignado con la fecha del torneo, asertando que la plataforma prohíba la acción mediante una excepción de conflicto de horarios.

9. PruebaEmpleadoPremioMetalico.java

Objetivo: Evitar que los empleados reciban premios en dinero.

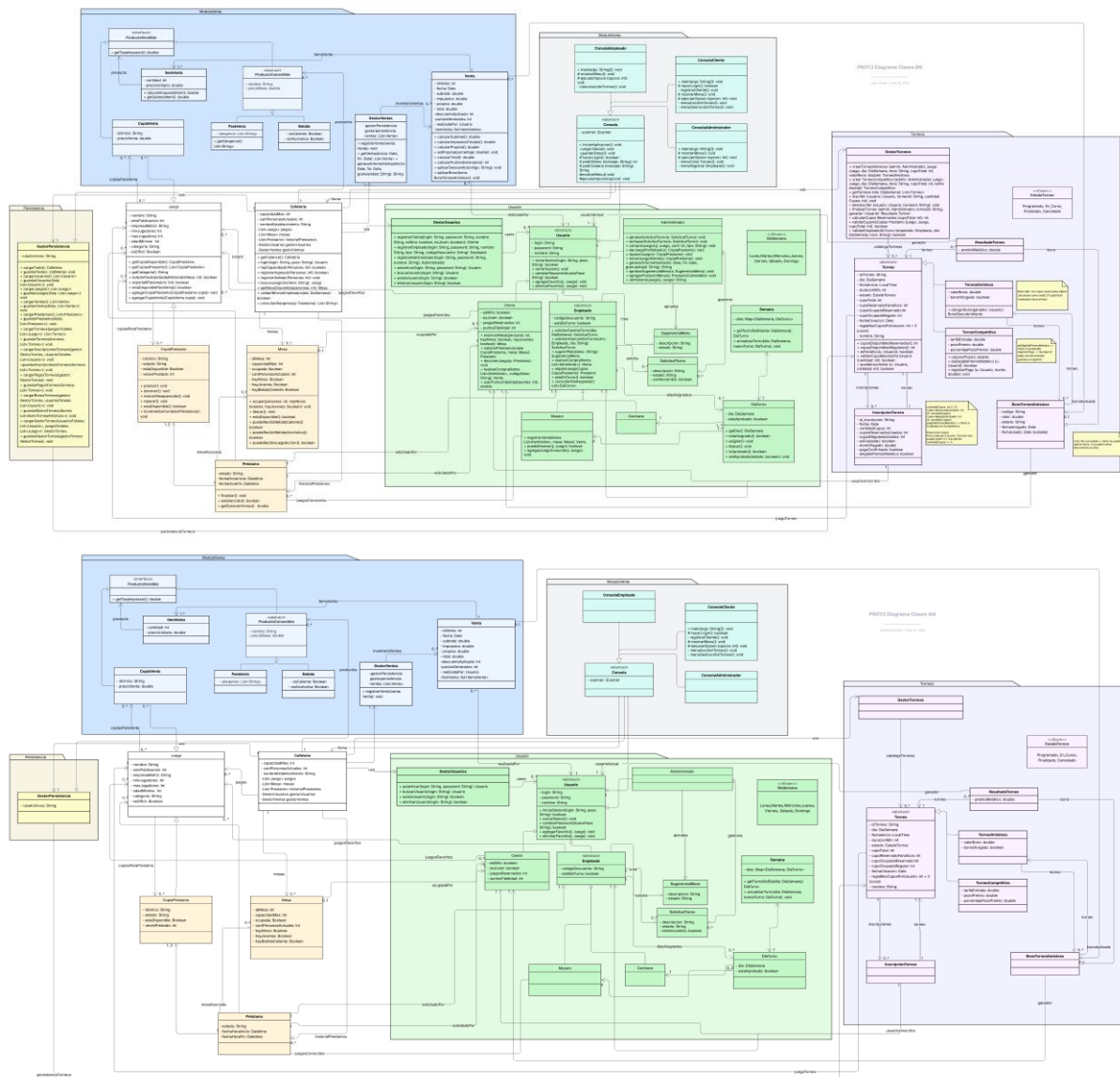
¿Qué hace? Simula a un empleado compitiendo y finalizando como ganador en un torneo. El test examina la bandera elegiblePrecioMetalico de la inscripción, la cual debe estar en false. Finalmente comprueba que al calcular su victoria, el premio monetario que ingresa a la cuenta del empleado sea estrictamente \$0.

10. PruebaBonoTorneoNoAcumulable.java

Objetivo: Garantizar que no se combinen múltiples descuentos de manera indebida.

¿Qué hace? Crea una compra a la cual primero se le aplica un bono ganado en un torneo amistoso. Luego, el cliente intenta aplicar adicionalmente un código de descuento de empleado. La prueba confirma que el sistema reemplaza o rechaza el descuento, imposibilitando que ambos descuentos se sumen para un mayor beneficio cruzado.

7. Diagramas de clase (alto nivel y normal)



Los archivos estan en el git.

8. Manual de Ejecución

Importar el proyecto en Eclipse.

Asegurarse de que la carpeta data/ esté en la raíz del proyecto (fuera de src).

Ejecutar Main.java.

Credenciales de prueba: Admin / admin123.